PYTHON FOR BEGINNERS · SESSION 11

Functions

Pack code into reusable named blocks: def, parameters, return values and scope.

Recap

Ask the class:

- In the marks manager, which code did we **repeat** for every command?
- What does `max(d, key=d.get)` give?
- Why did `int(input())` crash when the user typed a word?

What is a function?

A function is a **named block of code** that you can run **again and again**. You have already used many: `print()`, `len()`, `input()`, `sum()`.

- **Reuse:** write the code once, use it many times.
- **Tidy:** a long program becomes small named pieces.
- **Easy to fix:** a mistake is fixed in one place.

> **Ask**
> You write `len("hello")` without knowing how `len` works inside. Why is that useful?

Defining and calling

Use the keyword **`def`**, a **name**, round brackets, a **colon**, and an **indented body**.

functions.py

```
def greet():
    print("Hello!")
    print("Welcome to Python")

greet()
greet()
```

OUTPUT

```
Hello!
Welcome to Python
Hello!
Welcome to Python
```

- `def` only **defines** the function. Nothing runs yet.
- Writing **`greet()`** (with brackets) **calls** it, and the body runs.
- A function must be **defined before** it is called.

Forgetting the brackets

```
def greet():
    print("Hello!")

greet
print(type(greet))
```

OUTPUT

```
<class 'function'>
```

Careful

`greet` without brackets is just the **function itself**. It does **not** run.

Parameters and arguments

A **parameter** is a name inside the brackets of `def`. An **argument** is the real value you pass when you call.

```
def greet(name):
    print("Hello,", name)

greet("Aman")
greet("Riya")
```

OUTPUT

```
Hello, Aman
Hello, Riya
```

More than one parameter

```
def add(a, b):
    print(a + b)

add(3, 4)
add("Py", "thon")
```

```
OUTPUT
7
Python
```

- Arguments match the parameters **by position**: the first argument goes to the first parameter.
- The number of arguments must **match** the number of parameters.

Try it: wrong number of arguments

```
def add(a, b):
    print(a + b)

add(3)
```

```
ERROR
  File "your code", line 4, in <module>
TypeError: add() missing 1 required positional argument: 'b'
```

return

`return` **sends a value back** to the place where the function was called, and **ends** the function.

```
def square(n):
    return n * n

result = square(7)
print(result)
print(square(3) + square(4))
```

```
OUTPUT
49
25
```

print() is not return

```
def show_double(n):
    print(n * 2)

def get_double(n):
    return n * 2

a = show_double(5)
b = get_double(5)
print(a)
print(b)
```

OUTPUT

```
10
None
10
```

- `print()` only **shows** a value on the screen.
- `return` **gives** the value back so you can **keep using it**.
- A function with **no return** gives back **`None`**.

Ask

Why is `show_double(5) + 1` an error but `get_double(5) + 1` works?

return ends the function

Code **after** a `return` never runs. A function can have **several** `return` lines, but only **one** runs.

```
def sign(n):
    if n > 0:
        return "positive"
    elif n < 0:
        return "negative"
    return "zero"

print(sign(5), sign(-2), sign(0))
```

OUTPUT

```
positive negative zero
```

Returning many values

Returning several values with commas gives **one tuple**. Unpack it, like in Session 8.

```
def min_max(nums):
    return min(nums), max(nums)

print(min_max([4, 9, 2, 7]))
low, high = min_max([4, 9, 2, 7])
print(low, high)
```

OUTPUT

```
(2, 9)
2 9
```

Functions that answer yes or no

```
def is_even(n):
    return n % 2 == 0

print(is_even(10), is_even(7))
if is_even(4):
    print("4 is even")
```

OUTPUT

```
True False
4 is even
```

Default and keyword arguments

A parameter can have a **default value**. It is used when the caller does **not** give that argument.

```
def greet(name, greeting="Hello"):
    return greeting + ", " + name

print(greet("Aman"))
print(greet("Aman", "Good morning"))
```

OUTPUT

```
Hello, Aman
Good morning, Aman
```

Keyword arguments

You can name the argument when you call. The **order then does not matter**.

```
def power(base, exponent=2):
    return base ** exponent

print(power(5))
print(power(2, 10))
print(power(exponent=3, base=2))
```

OUTPUT

```
25
1024
8
```

Careful

Parameters **with** a default must come **after** those without one: `def f(a, b=1)` is fine, `def f(a=1, b)` is an error.

Built-in functions use this too

```
print("a", "b", "c", sep="-")
print("no new line", end="!")
print()
print(round(3.14159, 2))
```

OUTPUT

```
a-b-c
no new line!
3.14
```

Any number of arguments

A parameter with a **star** collects **all the extra arguments** into a **tuple**.

```
def total(*nums):
    print(nums)
    return sum(nums)

print(total(1, 2, 3))
print(total(10, 20))
print(total())
```

```
OUTPUT
(1, 2, 3)
6
(10, 20)
30
()
0
```

> **Note**
> This is the same `*` idea as star unpacking in Session 8. `print()` itself works this way.

Docstrings and good names

A string on the **first line** of a function describes what it does. It is called a **docstring**.

```
def average(marks):
    """Return the average of a list of marks."""
    return sum(marks) / len(marks)

print(average([90, 80, 70]))
print(average.__doc__)
```

```
OUTPUT
80.0
Return the average of a list of marks.
```

- Name a function by what it **does**: `average`, `is_even`, `get_grade`.
- Use **lowercase with underscores**, like variable names.
- Keep each function to **one job**.

Scope: local and global

Scope is where a variable can be **seen**. A variable made **inside** a function is **local**: it only exists while the function runs.

Try it: a local variable is not visible outside

```
def make():
    secret = 42
    print("inside:", secret)

make()
print("outside:", secret)
```

ERROR

```
inside: 42
  File "your code", line 6, in <module>
NameError: name 'secret' is not defined
```

- **Local:** made inside a function, gone when it ends.
- **Global:** made outside every function, can be **read** anywhere.
- Parameters are local variables too.

Reading a global

```
rate = 10

def price(qty):
    return qty * rate

print(price(5))
```

OUTPUT

```
50
```

Same name, different variable

If a function **assigns** to a name, Python makes a **new local** variable, even if a global with the same name exists. The global stays **unchanged**.

Work out the output first, then press Run to check

```
x = 10

def change():
    x = 99
    print("inside:", x)

change()
print("outside:", x)
```

The global keyword

To really change a global from inside a function, say **`global`** first.

```
count = 0

def add_one():
    global count
    count += 1

add_one()
add_one()
print(count)
```

OUTPUT

```
2
```

Better habit

Avoid `global`. Pass values **in** as arguments and send results **out** with `return`. The code is then easier to understand and test.

The better way

```
def add_one(count):
    return count + 1

count = 0
count = add_one(count)
count = add_one(count)
print(count)
```

OUTPUT

```
2
```

Lists and functions

A function gets the **same list**, not a copy. Changing it inside **changes it outside** too (lists are mutable). Numbers and strings cannot be changed this way.

```
def add_item(items):
    items.append("new")

def add_ten(n):
    n = n + 10

stuff = ["a", "b"]
add_item(stuff)
print(stuff)

x = 5
add_ten(x)
print(x)
```

OUTPUT

```
['a', 'b', 'new']
5
```

Passing a copy

```
def add_item(items):
    items.append("new")

stuff = ["a", "b"]
add_item(stuff.copy())
print(stuff)
```

OUTPUT

```
['a', 'b']
```

Functions calling functions

Small functions can be **combined**. Each one does one job.

```
def average(marks):
    return sum(marks) / len(marks)

def grade(avg):
    if avg >= 85:
        return "A"
    elif avg >= 70:
        return "B"
    elif avg >= 50:
        return "C"
    return "F"

def report(name, marks):
    avg = average(marks)
    return f"{name}: {avg:.1f} ({grade(avg)})"

print(report("Aman", [90, 80, 70]))
print(report("Riya", [85, 95, 88]))
```

OUTPUT

```
Aman: 80.0 (B)
Riya: 89.3 (A)
```

A function that calls itself

A function can call **itself**. This is called **recursion**. It needs a **stopping case**, or it never ends.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

OUTPUT

```
120
```

Back to the marks manager

Last class the commands were one **long block**. With functions each job gets a **name**, and the main loop becomes **short and clear**.

marks_manager.py (with functions)

```
students = {}

def add_marks(name, marks):
    students.setdefault(name, []).extend(marks)

def average(marks):
    return sum(marks) / len(marks)

def topper():
    return max(students, key=lambda n: average(students[n]))

def show_report():
    for name, marks in students.items():
        print(f"{name:<8}{sum(marks):>6}{average(marks):>9.1f}")

add_marks("Aman", [90, 80, 70])
add_marks("Riya", [85, 95, 88])
add_marks("Aman", [100])
show_report()
print("Topper:", topper())
```

OUTPUT

```
Aman       340     85.0
Riya       268     89.3
Topper: Riya
```

Ask

Which version is easier to **read**, last class or this one? Which is easier to **fix**?

A safe number reader

Remember `int(input())` crashing on a word? A function can read a number **until the user types a valid one**. This is for your computer, because it uses `input()`.

read_number.py (run on your computer)

```
def read_number(prompt):
    while True:
        text = input(prompt)
        if text.isdigit():
            return int(text)
        print("Please type a whole number")

age = read_number("Age: ")
print("Next year you will be", age + 1)
```

- `isdigit()` checks that the text is only digits.
- `return` ends the function **and the loop** as soon as the input is good.
- Use `read_number()` in the marks manager instead of `int(input())`.

Common mistakes

- Forgetting the **brackets** when calling: `greet` instead of `greet()`.
- Using **`print`** when you meant **`return`**, then getting `None`.
- Forgetting the **colon** or the **indentation** after `def`.
- Using a **local** variable outside its function.
- Calling a function **before** defining it.
- Giving the **wrong number** of arguments.
- A function with **no `return` on some paths** quietly gives `None`.

Try it: why does this print None?

```
def double(n):
    n * 2

print(double(4))
```

OUTPUT

```
None
```

Practice

Warm-up questions

1. Write a function `cube(n)` that **returns** the cube of a number.
2. Write a function `is_positive(n)` that returns `True` or `False`.
3. Write a function `greet(name, greeting="Hello")` and call it both ways.
4. Write a function that **returns the larger** of two numbers without using `max()`.
5. Write a function `count_vowels(text)` that returns the number of vowels.

Harder questions

6. Write `is_prime(n)` and use it to print all primes up to 50.
7. Write `fibonacci(n)` that returns a **list** of the first `n` Fibonacci numbers.
8. Write `reverse_words(sentence)` that returns the sentence with the words in **reverse order**.
9. Write `stats(nums)` that returns the **minimum, maximum and average** together.
10. Rewrite the **contact book** from last class using one function for each command.

Find the output

Work out the output first, then press Run to check

```
def f(a, b=2):
    return a * b

def g(items):
    items.append(0)

x = 3
print(f(x), f(x, 5), f(b=4, a=1))
nums = [1]
g(nums)
g(nums)
print(nums)
print(f(f(2)))
```

Try the even-numbers question

Your turn: finish evens() so it returns the even numbers of a list, then press Run

```
def evens(nums):
    # return a new list with only the even numbers
    pass

print(evens([1, 2, 3, 4, 5, 6]))
```

Homework and wrap-up

- Write a **calculator** with one function for each of `add`, `subtract`, `multiply` and `divide`, and a function that picks one by the operator typed.
- Write a function `is_palindrome(text)` and test it with 5 words.
- Rewrite the **marks manager** or the **contact book** so every command is a **function**.
- Write a function that takes a list and returns a **new list without duplicates**, keeping the order.
- Revise: **def**, **parameters**, **return**, **default arguments** and **scope**.

Next class

Files and error handling.